

Public Key Infrastructure (PKI)

Utworzyć folder **/pkilab** (w głównym katalogu dysku). Wszystkie dane zapisywać w tym folderze.

Zadanie 1 – Stworzenie root-CA

1. W folderze **/pkilab**, utworzyć strukturę katalogów dla CA opartego o pakiet OpenSSL. Utworzyć pliki **serial** i **index.txt** z zawartością odpowiednią dla nowego CA – odpowiednio **00 (dwie cyfry zero)** i **pusty plik (długości 0)**.
2. Wygenerować **klucz prywatny oraz certyfikat dla nowego CA**, ważne 10 lat. Jako **common name** podać: **grupaX CA**, gdzie X – numer stanowiska przy którym siedzimy.
Certyfikat i klucz prywatny wygenerować jednym poleceniem (**openssl req -x509 -newkey ...**).
3. Zmodyfikować odpowiednio domyślny plik konfiguracyjny **/etc/pki/tls/openssl.cnf** tak, aby wykorzystywał nowo utworzone katalogi i pliki.

PODPOWIEDŹ: W przypadku gdy nie wiemy, co znajduje się w jakimś pliku (certyfikat, żądanie, klucz, ...) warto wyświetlić jego zawartość komendą **cat** lub **less**.

Zadanie 2 – certyfikat kliencki

1. **Wygenerować klucz prywatny oraz żądanie** certyfikatu przeznaczonego do podpisania przez stworzone w poprzednim zadaniu CA. Jako **common name** podać: **grupaX**, gdzie X – numer naszego stanowiska.
2. **Podpisać żądanie** (z użyciem polecenia **openssl ca**), tworząc certyfikat z następującymi parametrami:
 - a. okres ważności certyfikatu 2 lata,
 - b. key usage: (1) szyfrowanie kluczy, (2) podpisywanie oraz (3) niezaprzeczalności (angl. NonRepudiation).**PODPOWIEDŹ:** Jeśli okaże się, że CA wymaga aby nazwa (common name) w certyfikacie klienta była taka sama jak w certyfikacie CA, to należy odpowiednio zmodyfikować politykę (policy) CA w pliku konfiguracyjnym.
3. Odczytać pełne informacje o podpisanym certyfikacie (poleceniem **openssl x509 ... -text**). Wynik polecenia przedstawić prowadzącemu i **wskazać elementy ilustrujące poprawne wykonanie zadania (nazwa wystawcy, nazwa właściciela, okres ważności, dozwolone użycia)**.

Zadanie 3 – format PKCS#12

1. Wygenerować paczkę PKCS#12 o nazwie **grupaX.p12** chronioną hasłem **lab142** i zawierającą:
 - a. Certyfikat kliencki i odpowiadający mu klucz prywatny, wygenerowane w zadaniu 2.
 - b. Certyfikat CA stworzonego w zadaniu 1. Aby go dołączyć do paczki, należy użyć opcji **-certfile plik**
2. Zapisać paczkę na serwerze FTP o adresie **winsrv.lab142.local** (poleceni: **ftp <adres>**, login: **anonymous**, hasło: **<brak>**). Po zalogowaniu należy najpierw wykonać polecenie: **binary**, a potem polecenie **put nazwa_pliku**. Jeśli ponownie chcemy przesłać plik o tej samej nazwie, należy najpierw usunąć poprzedni poleceniem **delete nazwa_pliku**.
3. Paczkę zachować do dalszych zadań.

Zadanie 4 – wydajność algorytmów symetrycznych

1. Skopiować plik tekstowy dostępny pod adresem <http://winsrv.lab142.local/pki/dane.txt> do swojego katalogu roboczego (najłatwiej poleceniem **wget <adres_url>**).
2. Skopiować plik tekstowy dostępny pod adresem <http://winsrv.lab142.local/pki/wyniki.txt> do swojego katalogu roboczego i umieszczać w nim wyniki pomiarów z pkt. 3.
3. Zweryfikować poprawność zaszyfrowywania i odszyfrowywania go z użyciem algorytmów **aes-256-cbc** oraz **des3**, z użyciem polecenia **openssl enc ...** (wykonać przy tym pomiary wartości wymienionych w pliku wyniki.txt).
UWAGA1: Obowiązkowo użyć do podania hasła opcji linii polecenia – nie podawać go interaktywnie z klawiatury.
UWAGA2: Jeśli otrzymujemy wyniki > 1 s. to wynik nie jest poprawny – prawdopodobnie: patrz uwaga 1.
4. Przesłać plik z wynikami na serwer FTP **winsrv.lab142.local** pod nazwą **wynikiX.txt** (X – numer naszego stanowiska) i zachować do dalszych zadań.

Zadanie 5 – podpisywanie danych z użyciem S/MIME

1. Podpisać **plik z wynikami z zad. 4** certyfikatem wygenerowanym w zadaniu 2. Wykorzystać polecenie **openssl smime** i stworzyć 2 wersje podpisanego pliku:
 - a. w pierwszej użyć czytelnej formy podpisanej wiadomości (bez opcji **-nodetach**),
 - b. w drugiej – formy zintegrowanej (z opcją **-nodetach**).
2. Porównać obie formy podpisanej wiadomości (ogólnie, wyświetlając je np. poleceniem **less plik**).
3. Zmodyfikować dowolną z 2 utworzonych, podpisanych wiadomości.
4. Zweryfikować podpisy. Sprawdzić, czy modyfikacja zostanie wykryta.
5. Zademonstrować poprawną oraz niepoprawną weryfikację prowadzącemu.

PODPowiedź: Do podpisania potrzeba: certyfikatu podpisującego (źródło danych o podpisującym, które będą umieszczone w dokumencie) i jego klucza prywatnego do samego wykonania podpisu.

Do weryfikacji potrzeba: certyfikatu podpisującego (bo zawiera klucz publiczny konieczny do sprawdzenia podpisu) oraz certyfikatu CA, które wystawiło certyfikat podpisującego (dla jego weryfikacji i sprawdzenia czy jest autentyczny).

Zadanie 6 – szyfrowanie danych z użyciem S/MIME

1. Zasyfrować plik z wynikami z zad. 4, z użyciem S/MIME, certyfikatu z zadania 2 i dowolnie wybranego algorytmu szyfrującego. W przypadku szyfrowania, nazwę pliku zawierającego certyfikat odbiorcy należy podać jako ostatni parametr polecenia (**`openssl smime -encrypt ... <plik_z_cert_odbiorcy>`**).
2. Zademonstrować prowadzącemu zasyfrowaną postać wiadomości oraz poprawne jej rozszyfrowanie (nie używać opcji **-out** przy rozszyfrowaniu – wyniki zostaną od razu wypisane na ekran).

PODPowiedź: Do zasyfrowania danych potrzeba klucza publicznego odbiorcy (a więc jego certyfikatu), a do odszyfrowania jego klucza prywatnego.

Zadanie 7 – przesłanie pliku

1. Pobrać certyfikat adresata tworzonej wiadomości z adresu <http://winsrv.lab142.local/pki/labpki.crt>. Jeśli przeglądarka nie pozwala pobrać tego pliku, należy użyć polecenia **`wget http://winsrv.lab142.local/pki/labpki.crt`**.
2. Z użyciem polecenia **`openssl smime`**: zasyfrować z użyciem pobranego certyfikatu adresata, a następnie podpisać własnym certyfikatem (stworzonym w zadaniu 2) **plik z wynikami** uzyskanymi w zadaniu 4. Plik wynikowy powinien mieć nazwę: **grupaX.enc**, gdzie X – numer naszego stanowiska.
3. Zapisać na serwerze FTP o adresie **winsrv.lab142.local** (login: **anonymous**, hasło: **<brak>**, polecenie: **`put nazwa_pliku`**):
 - a. zasyfrowany i podpisany plik utworzony w poprzednim punkcie 2: **grupaX.enc**
 - b. certyfikat swojego CA – pod nazwą: **grupaX.ca.cer**
 - c. swój certyfikat, którego użyto do podpisywania – pod nazwą: **grupaX.cer**

UWAGA: Nie da się podpisać i zasyfrować pliku jednym poleceniem. Nie można też jednocześnie czytać z pliku i pisać do niego – tzn. nie można podać tego samego pliku w **-in** i w **-out**.

Sugerowane nazewnictwo plików

	CA	Klient
Klucz prywatny	CA.key	klient.key
Żądanie certyfikatu	-	klient.req
Certyfikat	CA.cer	klient.cer

Popularny błąd:

- **polecenie „zawiesiło się”** – nie jest zawieszona, tylko oczekuje na brakujące dane. Trzeba je przerwać przez Ctrl+C i dodać do składni polecenia dodatkowy parametr.
- **wrong number of fields ...** – plik index.txt ma długość większą od 0 (nie jest pusty). Najlepiej stworzyć go poleceniem **`touch <nazwa_pliku>`**.
- **no such file or directory ... fopen('<nazwa pliku>)** – plik o podanej nazwie. NIE ZDARZA SIĘ, ABY POLECENIE NIE WIDZIAŁO ISTNIEJĄCEGO PLIKU – jeśli tak się wydaje to należy sprawdzić, w jakich folderach rzeczywiście mamy pliki (patrz niżej), a w jakich szuka ich polecenie (to jest podane w komunikacie).
Przydatne polecenia:
Linia poleceń: **`pwd`** – podaje pełną ścieżkę do aktualnego katalogu,
Środowisko graficzne: **prawy przycisk myszy na pliku->właściwości** – podaje ścieżkę w jakiej znajduje się wybrany plik.
- **expecting: TRUSTED CERTIFICATE** – w jednym z parametrów (lub pliku konfiguracyjnym) podaliśmy plik w którym powinien znajdować się certyfikat, a jest tam coś innego. Warto sprawdzić co, wyświetlając plik poleceniem **`cat nazwa_pliku`**.
- **expecting: CERTIFICATE REQUEST** – w jednym z parametrów (lub pliku konfiguracyjnym) podaliśmy plik w którym powinno znajdować się żądanie certyfikatu, a jest tam coś innego. Warto sprawdzić co, wyświetlając plik poleceniem **`cat nazwa_pliku`**.
- **Unable to load ... private key** - w jednym z parametrów podaliśmy plik w którym powinien znajdować klucz prywatny, a jest tam coś innego, lub też podaliśmy złe hasło do klucza prywatnego.
- **... certificate and ... private key do not match** – polecenie wymagało podania certyfikatu i klucza prywatnego, ale te które podano, nie tworzą pary.